

100 Days of Azure DevOps

Day 16 Handout — The bouncer at the commit club — ugly messages do not pass the rope

2 - Azure Repos & Git Mastery · Personal lab only · Educational content · Not a sales pitch

Architecture A — where the check runs (client vs server)

Hook / check	Runs on	Can be bypassed?
commit-msg (client)	git commit, reads \$1 message file	Yes: --no-verify
pre-commit (client)	Before the commit object exists	Yes: --no-verify
pre-push (client)	Before objects send to origin	Yes: --no-verify
pre-commit.com framework	Dev machine via .pre-commit-config.yaml	Yes if not installed
Azure Repos branch policy	Server: PR cannot complete	No (unless you are admin bypass)
Pipeline build validation	CI job on the PR	No if required on the branch
Server-side hooks on Azure Repos	Not your custom .git/hooks	Do not expect upload of commit-msg to ADO

Architecture B — decide client hook vs policy vs CI

Goal	Default	Escalate when
Conventional subject line	commit-msg hook in this lab	Team will skip --no-verify: add PR title check too
Secret scan	pre-commit detect-secrets / gitleaks locally	Also pipeline + never rely on honor system
Format / lint	pre-commit ruff/prettier	Same check in YAML so PRs from a phone still fail
Block force-push main	Server permission + policy (Day 17/19)	A client hook cannot stop another laptop
Commit prefix dayNN:	Optional lab hook	Do not require this on real product repos
Windows execution bit	hooks as .sample until copied	Git for Windows: hook must be executable / sh

Architecture C — pitfalls that make hooks folklore

Pitfall	What you see	Fix
Hook only in .git/hooks	Clone on laptop 2 has no bouncer	Commit a sample to docs/ + pre-commit config in Git
--no-verify as muscle memory	Bad messages still on main	Branch policy on PR title; CI required
CRLF breaks shebang	hook: command not found	Save LF; #!/bin/sh; Git Bash on Windows
Hook prints secrets on fail	PAT in the terminal scrollbar	Message = 'invalid format', not the file body
pre-commit without rev pin	Upstream hook changed behavior overnight	Pin sha in .pre-commit-config.yaml
Expecting Azure Repos to run your commit-msg	Server accepts anything	Policies + pipelines are the real gate

One-liner to remember

Hooks are the bouncer at the commit club — ugly messages do not get past the velvet rope.

Step-by-step lab (personal Azure / Azure DevOps only)

Personal clone. Client-side hook for this lab. Azure Repos will not run .git/hooks for you.

- 1 From repo root: copy the sample commit-msg to .git/hooks/commit-msg (Windows Git Bash: the hooks folder under .git).
- 2 On Windows, if the file is not executable, run the hook via Git Bash. Keep LF line endings.
- 3 git switch -c feature/day16-hooks && echo test > notes/day16.md && git add notes/day16.md
- 4 git commit -m "bad message" → expect the hook to reject (no type prefix).
- 5 git commit -m "docs: day 16 hook notes" → expect success.
- 6 Optional: add .pre-commit-config.yaml with repo: pre-commit-hooks, hook trailing-whitespace; pip install pre-commit && pre-commit install.
- 7 Push the markdown + any pre-commit config. Do not commit .git/hooks (it is not in the tree).

commit-msg hook — require conventional prefix (lab only)

```
#!/bin/sh
# save as .git/hooks/commit-msg (not committed)
grep -qE '^(docs|feat|fix|chore|test|refactor)(\(.+\))?: .+' "$1" || {
    echo "Commit message must look like: docs: short description"
    exit 1
}
```

Done checklist

- Can explain client hook vs branch policy vs pipeline check
- A bad commit message was rejected by commit-msg
- A conventional message succeeded and was pushed
- Understood --no-verify bypasses client hooks

Tomorrow — Day 17

Fork workflows & permissions